

Traffic Sign Recognition

Classification & Object Detection with TensorFlow, Keras & KerasCV

German Traffic Sign Detection Benchmark (GTSDB)

Deliverable:	a single Jupyter notebook
Group size:	2 students
Assessment:	Notebook + oral defense
Dataset:	GTSDB (downloaded by the notebook)

Group Work Policy

This assignment is completed in groups of 2 students. Both students are equally responsible for all parts of the notebook. During the oral defense, each student may be questioned on any section. Divide work for efficiency, but review everything together.

Introduction & Learning Objectives

This assignment asks you to build a complete traffic sign recognition pipeline from scratch using the German Traffic Sign Detection Benchmark (GTSDDB). You will progress from simple image classification on clean cropped images, through the more realistic challenge of multi-label classification on full road scenes, and finally to full object detection. Every step uses the same dataset, so you will directly observe how the same underlying data presents very different modeling challenges depending on the task formulation.

By the end of this assignment you should be able to:

- Design, train, and evaluate convolutional neural networks from scratch for multi-class classification.
- Select and apply transfer learning strategies.
- Understand and implement multi-label classification — recognizing that it is fundamentally different from multi-class classification.
- Build an object detection pipeline using KerasCV, adapting it to the specific properties of your dataset.
- Apply data augmentation appropriately for both classification and detection tasks.
- Evaluate models with appropriate metrics and analyze failure cases critically.

Dataset

Overview — GTSDDB

The German Traffic Sign Detection Benchmark (GTSDDB) contains 900 full road-scene images (1360×800 pixels) captured near Bochum, Germany in spring and autumn 2010, across urban, rural, and highway environments at varying times of day. The dataset was originally introduced as a competition at IJCNN 2013.

Property	Detail
Total images	900 road scene images (600 train / 300 test, original split)
Resolution	1360×800 px, PPM format (convert to PNG — see § 2.3)
Annotated images	741 images contain at least one sign; 159 are sign-free
Total sign instances	1 213 annotated bounding boxes across all images
Signs per image	0 to 6 (including images with no signs at all)
Fine-grained classes	43 individual sign types (class IDs 0–42)
Super-classes	4 groups: Prohibitory, Danger, Mandatory, Other (IDs are not contiguous within groups — see § 2.5)
Sign bounding box size	15×15 px (distant sign) up to 250×250 px (close sign)

Table 1: GTSDDB dataset properties.

Downloading the Dataset

Download the official dataset directly from the INI Benchmark server. No account or API key is required. Note: the archive also contains several sub-directories of pre-cropped individual signs: **ignore these (delete them)**. Use only the full scene PPM images (`00000.ppm` – `00899.ppm`) and `gt.txt`.

[FullIJCNN2013.zip](#)

```
# Download FullIJCNN2013.zip from:
# https://sid.erda.dk/public/archives/ff17dc924eba88d5d01a807357d6614c/published-
  archive.html

# Unzip the archive. The extracted folder contains:
# 00000.ppm - 00899.ppm    full road scene images
# gt.txt      ground truth annotations
# ReadMe.txt  class ID descriptions
```

Converting PPM Images to PNG

Required step — TensorFlow cannot read PPM files

TensorFlow's image I/O functions (`tf.io.read_file` + `tf.image.decode_image`) do not support the PPM format. You must convert all 900 images to PNG **before** building any `tf.data` pipeline. Pillow handles PPM natively and writes lossless PNG files, so no quality is lost.

Run this cell once, immediately after unzipping the dataset. It is safe to re-run: the `if not png.exists()` guard skips files that have already been converted.

```
from pathlib import Path
from PIL import Image

img_dir = Path('FullIJCNN2013') # folder that contains the .ppm files

ppm_files = sorted(img_dir.glob('*.ppm'))
for ppm in ppm_files:
    png = ppm.with_suffix('.png')
    if not png.exists():
        Image.open(ppm).save(png)

print(f'Done. {len(ppm_files)} PPM files converted to PNG.')
```

After conversion, use the `.png` extension everywhere in your pipeline. When loading annotations from `gt.txt` (which lists `.ppm` filenames), replace the suffix on the fly:

```
import pandas as pd

gt = pd.read_csv('FullIJCNN2013/gt.txt', sep=';',
               names=['filename', 'x1', 'y1', 'x2', 'y2', 'class_id'])

# Replace .ppm with .png in the filename column
gt['filename'] = gt['filename'].str.replace('.ppm', '.png', regex=False)
print(gt.head())
```

Annotation Format

The file `gt.txt` contains one line per annotated sign instance. There are **1 213 lines** in total, one per bounding box:

```
# Each line has the form:
# filename ; x1 ; y1 ; x2 ; y2 ; class_id
# Example:
# 00073.ppm ; 241 ; 355 ; 305 ; 419 ; 38
```

Class ID to Super-Class Mapping

The 43 fine-grained class IDs are grouped into 4 super-classes. The grouping is defined by the label in parentheses in `ReadMe.txt` and is **not** a simple contiguous range. Verify the mapping below against your own copy of `ReadMe.txt` before writing any code.

Super-class	Class IDs	Examples	# instances
Prohibitory (0)	0–5, 7–10, 15, 16	Speed limits (20–120 km/h), no overtaking, no trucks, no traffic both ways	557
Danger (1)	11, 18–31	Priority at intersection, bend, uneven road, slippery road, pedestrian/school/cycles crossing, construction, snow, animals	219
Mandatory (2)	33–40	Go right/left/straight, keep right/left, roundabout	163
Other (3)	6, 12–14, 17, 32, 41, 42	Restriction ends, priority road, give way, stop, no entry, direction/information signs	274
Total	0–42 (43 IDs)		1 213

Table 2: Super-class definitions, exact class IDs, and instance counts (from `gt.txt`).

Sign Patch Extraction (shared preprocessing)

Before starting the notebook, you must extract individual sign crops from the full road scene images. These cropped patches are used in Tasks 1 and 2 (classification). Tasks 3 and 4 use the full road scene images directly.

Cropping Procedure

- For each row in `gt.txt`, open the corresponding **PNG** image (after conversion) and crop the region defined by (x_1, y_1, x_2, y_2) .
- Resize each crop to exactly 96×96 pixels.
- Save the cropped patch as a PNG file.

Output File Naming Convention

Saved patch filenames must follow this exact convention:

```
# If image 00042.ppm contains 3 signs, save patches as:
# 00042_1.png    (first sign in the file, top-to-bottom order)
# 00042_2.png    (second sign)
# 00042_3.png    (third sign)

# General rule:
# {original_basename}_{index}.png
# where index starts at 1 and increments for each sign in that image
```

Suggested Extraction Code Structure

```

import cv2
import numpy as np, pandas as pd
from pathlib import Path

gt = pd.read_csv('FullIJCNN2013/gt.txt', sep=';',
                 names=['filename', 'x1', 'y1', 'x2', 'y2', 'class_id'])

# Correct super-class mapping derived from ReadMe.txt
# 0=Prohibitory 1=Danger 2=Mandatory 3=Other
_MAP = [
    0, 0, 0, 0, 0, 0, 3, 0, 0, 0, 0,  # IDs 0-10
    1, 3, 3, 3, 0, 0, 3, 1, 1, 1,  # IDs 11-20
    1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  # IDs 21-31
    3, 2, 2, 2, 2, 2, 2, 2, 3, 3  # IDs 32-42
]
class_to_superclass = np.array(_MAP)

out_dir = Path('patches')
out_dir.mkdir(exist_ok=True)

counter = 0
prev = ''

for _, row in gt.iterrows():
    fbase = row.filename[:-4]          # strip .ppm
    if fbase != prev:
        counter = 0
        prev = fbase
    png_path = Path('FullIJCNN2013') / (fbase + '.png')
    img = cv2.imread(str(png_path))
    if img is None:
        continue
    crop = img[int(row.y1):int(row.y2), int(row.x1):int(row.x2)]
    if crop.size == 0:
        continue
    patch = cv2.resize(crop, (96, 96), interpolation=cv2.INTER_AREA)
    counter += 1
    cv2.imwrite(str(out_dir / f'{fbase}_{counter}.png'), patch)

print('Extraction complete.')

```

Check before continuing

After extraction, verify that your patch count matches the total number of annotated bounding boxes in `gt.txt`: **1 213 patches** in total. Count files in your `patches/` folder and report this number in your notebook. If the count differs, there is a bug in your extraction code — do not proceed to Task 1 until this is fixed.

Classification Tasks

This section contains three classification tasks of increasing complexity. Tasks 1 and 2 operate on the 96×96 cropped patches you extracted in Section 3. Task 3 switches to full road scene images and introduces a fundamentally different problem formulation.

Data Splits

Define your train / validation / test split once, at the top of the notebook, and reuse it across all four tasks. A stratified 70/15/15 split across the patch dataset is recommended. Document your split strategy and random seed in a dedicated cell.

T1 CNN from Scratch — 4 Super-classes

Objective

Design and train a convolutional neural network entirely from scratch to classify sign patches into the 4 super-classes (Prohibitory, Danger, Mandatory, Other). The goal of this task is to give you full control over the architecture and to establish a baseline before using any pre-trained features.

Requirements

- Build a CNN using only `tf.keras` layers (`Conv2D`, `MaxPooling2D`, `BatchNormalization`, `Dropout`, `Dense`). Do not load any pre-trained weights.
- You choose the number of layers, filter sizes, and other hyperparameters. Justify your choices briefly in a markdown cell.
- Implement a data augmentation pipeline using `KerasCV` or `tf.keras` preprocessing layers. You decide which transformations to apply — justify your choices based on what types of variation are realistic for traffic sign images.
- Train using an appropriate optimizer, loss function, and learning rate schedule. Document all choices.
- Plot training and validation loss and accuracy curves for every experiment.

Evaluation

- Report overall accuracy on the test set.
- Show a non-normalized confusion matrix.
- Plot ROC curves per class.
- You are free to include additional evaluation metrics of your choice (e.g. mean AUC of ROC per class, per-class F1-score, precision-recall curves).
- Visualize at least 8 correctly classified and 8 misclassified patches, with predicted and true labels.
- Briefly analyze which super-classes are most often confused and propose a reason.

T2 Pre-trained CNN — 43 Classes

Objective

Train a classification model using a pre-trained backbone to classify sign patches into all 43 individual sign classes. You choose the backbone architecture.

Note — class imbalance

The 43 classes are highly imbalanced in GTSDDB. Before training, plot the patch count per class and apply at least one mitigation strategy (e.g. class weights, oversampling, or targeted augmentation of rare classes). Document which strategy you chose and why.

Requirements

- Select a pre-trained backbone available in `tf.keras.applications` (e.g. EfficientNetV2, ConvNeXt, MobileNetV3).
- Use a two-phase training strategy: (1) freeze the backbone and train the classification head; (2) unfreeze some or all backbone layers and fine-tune with a reduced learning rate. Document the number of layers you unfreeze and why.
- Add a classification head suited for 43-class single-label classification (43-unit softmax output). Document your design choices.
- Apply data augmentation. You may reuse, extend, or modify your earlier pipeline.
- Plot loss and accuracy curves for both training phases.

Evaluation

- Report overall accuracy and macro F1-score on the test set across all 43 classes.
- Show a per-class F1-score bar chart (a 43×43 confusion matrix is too dense to read — use the chart instead). Highlight the five worst-performing classes.
- Produce a comparison table across both tasks (Task 1 and Task 2): accuracy, macro F1, parameter count, approximate training time.
- Discuss: how much did transfer learning help compared to Task 1? Which of the 43 classes benefited most and which are still confused?

Deliverable — The Notebook (Tasks 1 & 2)

Patch extraction code	Working code from Section 3 producing correctly named 96×96 PNG patches
Task 1 — 4 classes	CNN from scratch, augmentation pipeline, training curves, confusion matrix, error analysis
Task 2 — 43 classes	Pre-trained backbone, two-phase fine-tuning, class imbalance mitigation, per-class F1 bar chart, comparison table
Markdown documentation	All architecture and hyperparameter choices justified in markdown cells

T3 Multi-label Classification on Full Road Scene Images

Read this section carefully — the task formulation is different

Tasks 1 and 2 are multi-class classification problems: each image contains exactly one sign and belongs to exactly one class. Task 3 is a multi-label classification problem: each full road scene image may contain zero, one, or multiple signs, potentially from multiple super-classes at the same time. This requires a different output layer, loss function, and evaluation approach.

Why This Is Multi-label, Not Multi-class

Multi-class (Tasks 1–2)	Multi-label (Task 3)
Each image belongs to exactly one class	Each image can belong to zero or more classes
Output: softmax over N classes — probabilities sum to 1	Output: sigmoid applied independently to each of N outputs — probabilities are independent
Loss: categorical (or sparse categorical) cross-entropy	Loss: binary cross-entropy applied independently to each output
Prediction: argmax of output vector	Prediction: threshold each output independently (sigmoid $> 0.5 \Rightarrow$ class present)
Example: patch $\Rightarrow$ class Prohibitory	Example: road scene $\Rightarrow$ classes {Prohibitory, Danger}

Preparing Labels for Task 3

```
labels = np.zeros((len(image_files), 4), dtype=np.float32)
for i, fname in enumerate(image_files):
    for _, row in gt[gt.filename == fname].iterrows():
        sc = class_to_superclass[row.class_id]
        labels[i, sc] = 1.0
```

Objective

Train a model to predict which super-classes of traffic signs are present in a full road scene image. Given a 1360×800 road scene, the model should output a probability for each of the 4 super-classes independently. A single image may activate zero, one, two, or all four super-class outputs simultaneously.

Requirements

- Select a pre-trained backbone (your choice — it may be the same as Task 2 or different). Justify your selection for this new task and input size.
- Add an appropriate multi-label classification head. The final layer must use sigmoid activation, not softmax.
- Use binary cross-entropy as the loss function.
- Resize full scene images to an appropriate input resolution (your choice — document the trade-off between resolution and memory/speed).
- Apply data augmentation suitable for full scene images. Note: augmentations like extreme rotation are less appropriate here than for patches.

- Train the model and plot loss curves. Monitor both loss and a suitable metric (e.g. binary accuracy or F1 per label).

Evaluation

- For each super-class output, report precision, recall, and F1-score at a threshold of 0.5.
- Plot threshold-independent curves for each class (e.g. ROC).
- Report the overall micro-averaged and macro-averaged F1.
- Experiment with at least two different classification thresholds (e.g. 0.3, 0.5, 0.7) and report the effect on precision and recall for each super-class.
- Visualize at least 10 predictions: show the full road scene image, the ground-truth label vector, and the predicted probability vector.
- Include at least 3 false positive and 3 false negative examples with analysis.

Reflection question (mandatory, in a markdown cell)

Multi-label classification on full images tells you *what* is present, but not *where*. Write a short paragraph (5–8 sentences) in your notebook explaining why this limitation matters for a real-world road safety application, and why object detection (Section 5) is the appropriate next step.

Update Deliverable to include Task 3

The deliverable is the complete notebook including Task 3 and 4 (next task). Make sure your notebook runs from top to bottom without errors before submitting.

Object Detection Task

T4 Object Detection with KerasCV

This section builds a traffic sign detection system that localizes and classifies signs in full road scene images. Unlike Tasks 1–3, the model must simultaneously predict *where* signs are (bounding boxes) and *what* they are (4 super-class labels). It must also correctly predict nothing on the 159 images that contain no signs.

Why detection is harder than multi-label classification

Task 3 could tell you a Prohibitory sign is somewhere in the image. Detection must tell you exactly *where* — and do so for every sign in the image, at any scale, against a cluttered road background. This requires a fundamentally different architecture and training approach.

Framework

All detection work in this notebook must use the KerasCV object detection API. You may choose between `RetinaNet` and `YOLOV8Detector` — both are available in KerasCV. State your choice at the top of the notebook and briefly justify it.

Annotation Preparation

- Load the full 900 road scene PNG images and their bounding box annotations from `gt.txt`.
- Map class IDs to the 4 super-class indices using `class_to_superclass` (defined in Section 3 and reproduced in § 2.5).
- Convert annotations to the KerasCV `bounding_boxes` format:

```
bboxes = {
    'boxes': tf.constant([[x1, y1, x2, y2]], dtype=tf.float32),
    'classes': tf.constant([superclass_id], dtype=tf.float32),
}

bboxes_empty = {'boxes': tf.zeros([0, 4]), 'classes': tf.zeros([0])}
```

- Resize all images to a consistent resolution before training (e.g. 640×640 or 512×512). You choose and justify.
- Use the same 70/15/15 train/validation/test split (applied to the 900 full images).

Problem Adaptation (required section — do not skip)

Before building the model, you must analyze the dataset properties that will affect your design decisions. This section must appear in your notebook with results and discussion.

- **Bounding box analysis:** plot histograms of bounding box width, height, and aspect ratio across the training set. Compute the 10th, 50th, and 90th percentiles of box width and height (in pixels, after resizing to your chosen input resolution). Present these as a table.
- **Anchor design:** based on your bounding box analysis, define anchor scales and aspect ratios appropriate for GTSDDB traffic signs. Justify your choices with reference to your plots. Compare at least two anchor configurations.
- **Negative sample strategy:** explain how you handle the 159 sign-free images during training. Do you include all of them? Subsample? Justify your decision.

- **Scale challenge:** traffic signs range from 15×15 to 250×250 pixels. Describe what feature pyramid levels you expect to be most important and why.

Data Augmentation for Detection

Augmentation for detection must preserve bounding box coordinates. Use KerasCV geometry-aware layers (such as `JitteredResize`, `RandomFlip`, `RandomShear`). You choose which transformations to apply — justify your selections.

Mandatory check before training

Visualize at least one batch of augmented images with their transformed bounding boxes drawn on top. If bounding boxes do not align correctly with the augmented content, there is a bug in your pipeline. Do not proceed to training until this check passes.

Model Build and Training

- Initialize the backbone with ImageNet pre-trained weights. The detection head is randomly initialized.
- Apply a two-phase training strategy: (1) freeze the backbone, train the detection head; (2) unfreeze part of the backbone and fine-tune. Document the number of epochs and learning rates for each phase.
- Use KerasCV's built-in detection loss (focal loss + smooth L1). Do not implement the loss from scratch.
- Monitor validation mAP@0.5 throughout training. Save the best checkpoint.
- Plot training loss and validation mAP curves for both phases.

Evaluation

- Reload the best saved checkpoint for all evaluation — do not evaluate the final epoch weights.
- Report mAP@0.5 and mAP@0.5:0.95 on the test set, both overall and per super-class.
- Plot a Precision-Recall curve for each super-class.
- Visualize at least 20 test images with ground-truth boxes (draw in green) and predicted boxes (draw in red), labeled with class name and confidence score.
- Report the false positive rate on the 159 sign-free images (i.e. what fraction of negative images produce at least one prediction above your confidence threshold).
- Report inference time per image on your hardware.

Failure Analysis

Identify and present at least 6 failure cases, covering at least two of the following failure types:

- Missed detection (sign present, not found)
- False positive (no sign, but a detection was produced)
- Wrong class (sign found at correct location but misclassified)
- Partial detection (only some signs in a multi-sign image are found)

For each failure case: show the image, describe the failure, and propose a specific reason and a potential remedy.

Comparison: Frozen Backbone vs. Fine-tuned

Present a comparison table with the following columns:

Model variant	mAP@0.5	mAP@0.5:0.95	FP-rate	Time
Frozen backbone (phase 1 only)				
Fine-tuned (phase 1 + 2)				

Table 3: Detection results: frozen vs. fine-tuned backbone.

Discuss the results: did fine-tuning help? By how much? Was the improvement worth the extra training time?

Deliverable — Object Detection (Task 4)	
Annotation preparation	Correct KerasCV format, resolution choice justified, negative images handled
Problem adaptation	BBox analysis plots, anchor design with justification, negative strategy, scale discussion
Augmented bbox visualization	Pre-training check: at least one batch shown with transformed boxes
Two-phase training	Loss and mAP curves for both phases, checkpoint saving
Full evaluation	mAP@0.5, mAP@0.5:0.95, per-class PR curves, 20+ prediction visualizations
Failure analysis	6+ failure cases with discussion across at least 2 failure types
Comparison table	Frozen vs. fine-tuned comparison with discussion

Submission Requirements

What to Submit

- `XXXXXXXXXXXX_projeto.ipynb` — Tasks 1–4. Replace `XXXXXX` with your student numbers.
- The notebook must run completely from top to bottom on a fresh kernel with no errors.
- All output cells must be pre-rendered (do not clear outputs before submitting).
- Do not include the raw GTSDDB images or your patch folder in the submission. Your notebook must download and set up the data in its first cell.

Notebook Structure Requirements

- **First cell:** a markdown cell with group number, both student names and IDs, submission date, and a one-paragraph abstract.
- **Second cell:** a setup/install cell that downloads the GTSDDB data, converts PPM to PNG, and installs any non-standard packages.
- Each task must begin with a clearly labeled markdown heading (e.g. `## Task 1 -- CNN from Scratch -- 4 Classes`).
- All architecture choices, hyper-parameter decisions, and augmentation selections must be explained in markdown cells immediately before the relevant code.
- All plots must have titles, axis labels, and legends where applicable.

Reproducibility

- Set all random seeds at the top of each notebook: `tf.random.set_seed(42), np.random.seed(42)`.
- Document the exact package versions used (run `pip freeze` or print version strings in a cell).

- If you train on GPU (recommended), note the GPU type used.

Oral Defense

Each group will present their work in a short oral defense session. The session will last approximately 20 minutes per group and consists of a brief walkthrough of the submitted notebooks followed by questions from the examiner.

Format

- Both students must be present. Both may be questioned on any part of the notebook.
- No separate presentation slides are required — the notebooks are your presentation.
- The examiner may ask you to scroll to any cell in the notebook and explain it.
- You may be asked to modify a cell or describe what would happen if a hyperparameter were changed.

Sample Oral Questions

The following are examples of the type of questions you may be asked. This is not an exhaustive list.

1. Why did you choose this augmentation strategy for the patch classifier? Would you use the same augmentations for detection?
2. Walk us through your anchor design decision. What would happen if you used COCO default anchors?
3. Your model has a 15% false positive rate on sign-free images. How would you reduce this?
4. How did fine-tuning change the detection results? Which layers were most important to unfreeze?
5. Pick one failure case from your notebook. Why did the model fail here, and what would fix it?

Academic Integrity

All submitted code must be original work by the two group members. Use of public tutorials to understand concepts is permitted, but submitted code may not be copied from external sources. Any adapted code must be cited with a comment. AI tools may be used for boilerplate only — all model design, training decisions, analysis, and written discussion must be your own. Both group members sign the submission, confirming that both contributed to and understand all parts.